Ansible Automation - Use Cases

A reference of the use cases by which Ansible Automation Platform is commonly implemented, by domain. Each use case is described in the same format (Description, Collection, Key content, Notes) and mapped to Red Hat Certified and Validated content.

10 solution use cases · 38 product use cases · generated 2026-10-21 · community reference, not an official Red Hat publication

Contents

1. Linux (RHEL & Satellite)
    - Solution use cases (10)
        - Build golden images with image builder
        - Install and maintain Satellite and Capsules
        - Configure Satellite as code
        - Provision RHEL with Satellite
        - Patch RHEL at scale
        - Standardise RHEL configuration
        - Manage users, sudo and SSH
        - Remediate with Red Hat Insights
        - Convert CentOS and Oracle Linux to RHEL
        - Upgrade RHEL in place with snapshot rollback
    - Product use cases
        - POSIX and Linux systems (3)
        - Red Hat convert2rhel (1)
        - RHEL in-place upgrade (Leapp) (2)
        - Linux LVM (1)
        - RHEL image builder (osbuild) (2)
        - Red Hat Satellite (4)
        - Windows Subsystem for Linux (1)
        - OpenSVC (2)
        - OpenSVC (2)
        - Red Hat Insights (1)
        - RHEL in-place upgrade (Leapp) (2)
        - Red Hat Enterprise Linux (6)
        - Red Hat Satellite (Foreman and Katello) (10)
        - Red Hat Satellite (1)

Linux (RHEL & Satellite)

Build, provision, patch, standardise, convert and upgrade Linux estates.

Solution use cases

Use Case: Build golden images with image builder

Description	Replace hand-built "gold" images with images that are defined as code, composed on a schedule and published to a known location. Every image is traceable to a blueprint in Git, so a rebuild after a security fix is one job run instead of a manual exercise.
Collection	`infra.osbuild` (Validated)
Key content	• `infra.osbuild.setup_server` — Setup osbuild compose server • `infra.osbuild.builder` — The main entry point for the builder role. • `infra.osbuild.create_blueprint` — Create a new blueprint file • `infra.osbuild.push_blueprint` — Push a blueprint file into composer • `infra.osbuild.start_compose` — Start an ostree compose • `infra.osbuild.wait_compose` — Wait for a compose to complete • `infra.osbuild.export_compose` — Export successfully composed image from osbuild-composer service • `infra.osbuild.inject_ks` — Inject Kickstart into composed ISO image.
Notes	• The image types that the collection documents as tested are the RHEL for Edge types (`edge-commit`, `edge-container`, `edge-installer`, `edge-simplified-installer`, `edge-raw-image`). See Build and update edge OS images for the edge-specific flow. • Image-based installs pair with Provision RHEL with Satellite, which uses the same kickstart and host group concepts. • Keep post-install configuration out of the image where possible and apply it with Standardise RHEL configuration after first boot, so one image serves many roles.

Use Case: Install and maintain Satellite and Capsules

Description	Red Hat Satellite is the content, patch and provisioning hub for a RHEL estate. This use case automates the server side of that hub: installing Satellite and its Capsules, loading the subscription manifest and setting up the routine operations (backups, metrics, health checks) so every site is built the same way.
Collection	`redhat.satellite_operations` (Certified), `redhat.satellite` (Certified), `redhat.rhel_system_roles` (Certified)

Key content	• `redhat.satellite_operations.installer` — Run the satellite-installer • `redhat.satellite_operations.capsule_certs_generate` — Generates a certificate bundle for Satellite Capsule scenario • `redhat.satellite_operations.backup` — Sets up backup cronjob for Satellite • `redhat.satellite_operations.metrics` — Setup Performance Co-Pilot and Grafana to gather metrics of a Satellite installation • `redhat.satellite_operations.cloud_connector` — Install and configure Red Hat Cloud Connector • `redhat.satellite.manifest` — Fetch a Manifest from the Red Hat Portal and upload it to Foreman • `redhat.satellite.smart_proxy` — Manage Capsules • `redhat.satellite.status_info` — Get status info
Notes	• The next step is Configure Satellite as code, which defines organizations, products, content views and activation keys. • Test restores from the backup periodically; a backup that was never restored is not proof of recovery. • `redhat.satellite_operations` has no dedicated upgrade role, so run Satellite version upgrades from a separate job template and keep a fresh backup before each one.

Use Case: Configure Satellite as code

Description	Keep the configuration of Red Hat Satellite (organizations, locations, products, repositories, content views, lifecycle environments, activation keys, host groups) as YAML in Git. The same definitions can then build a new Satellite, rebuild a lost one or promote settings from a test server to production.
Collection	`infra.satellite_configuration` (Validated), `redhat.satellite` (Certified)
Key content	• `infra.satellite_configuration.filetree_create` — Role to export the Satellite's configuration to get Configuration as Code • `infra.satellite_configuration.filetree_read` — Read the Satellite Configuration as Code definition files • `infra.satellite_configuration.dispatch` — Role to apply the already loaded Configuration as Code into Satellite • `infra.satellite_configuration.filetree_reconcile` — Reconcile live Satellite configuration against Configuration as Code • `redhat.satellite.organizations` — Create and manage Organizations • `redhat.satellite.lifecycle_environments` — Create and manage Lifecycle Environments • `redhat.satellite.content_views` — Create and manage Content Views • `redhat.satellite.activation_keys` — Create and manage Activation Keys
Notes	• Compute resources, compute profiles, global parameters, SCAP content and policies, job templates, webhooks and HTTP proxies are listed as not yet covered end to end by `infra.satellite_configuration`. Manage them with the `redhat.satellite` modules. • The collection includes filters that drop locked (built-in) Satellite objects, so the export is meant to hold your own objects only. Review the first export before committing it. • This tree defines the content that Patch RHEL at scale and Provision RHEL with Satellite rely on.

Use Case: Provision RHEL with Satellite

Description	Turn a request for a server into a repeatable build. Satellite holds the provisioning building blocks (subnets, domains, installation media, partition tables, templates, compute resources and host groups); Ansible creates the host from them, waits for the build and hands it to the configuration playbooks.
Collection	`redhat.satellite` (Certified), `redhat.rhel_system_roles` (Certified)
Key content	• `redhat.satellite.host` — Manage Hosts • `redhat.satellite.hostgroup` — Manage Hostgroups • `redhat.satellite.compute_resource` — Manage Compute Resources • `redhat.satellite.provisioning_template` — Manage Provisioning Templates • `redhat.satellite.subnet` — Manage Subnets • `redhat.satellite.host_power` — Manage Power State of Hosts • `redhat.satellite.wait_for_task` — Wait for a task • `redhat.satellite.registration_command` — Manage Registration Command
Notes	• Pair the post-build step with Standardise RHEL configuration so every host leaves provisioning compliant. • Image-based provisioning uses images from Build golden images with image builder or from the compute platform; define them with `redhat.satellite.image`. • Capture all building blocks in Git with Configure Satellite as code. • Use a survey or an ITSM request as the trigger so the requester supplies only the name, size and role.

Use Case: Patch RHEL at scale

Description	Patch every RHEL host from content that has been tested, not from whatever is newest. Satellite freezes the package set in content view versions and promotes it through lifecycle environments (for example Dev, Test, Production); Ansible publishes, promotes and applies the patches in a controlled order.
Collection	`redhat.satellite` (Certified)
Key content	• `redhat.satellite.lifecycle_environment` — Manage Lifecycle Environments • `redhat.satellite.content_view` — Manage Content Views • `redhat.satellite.content_view_publish` — Publish a list of Content Views • `redhat.satellite.content_view_version` — Manage Content View Versions • `redhat.satellite.host_errata_info` — Fetch information about Host Errata • `redhat.satellite.job_invocation` — Invoke Remote Execution Jobs • `redhat.satellite.sync_plan` — Manage Sync Plans • `redhat.satellite.content_view_version_cleanup` — Clean up unused Content View Versions
Notes	• Build the whole cycle as a workflow: sync, publish, promote to Dev, patch Dev, approval, promote to Test, and so on. • Take LVM snapshots before patching critical hosts with the roles described in Upgrade RHEL in place with snapshot rollback. • Use Remediate with Red Hat Insights to find and fix vulnerabilities, and this use case for planned, bulk patching. • Start the lifecycle design in Configure Satellite as code so content views and environments are versioned.

Use Case: Standardise RHEL configuration

Description	Define the standard operating environment (SOE) once as variables and apply it to every RHEL host. RHEL System Roles give a supported, version-independent interface for time, firewall, SELinux, logging, crypto and kernel settings, so one playbook can target several RHEL releases.
Collection	`redhat.rhel_system_roles` (Certified), `ansible.posix` (Certified)
Key content	• `redhat.rhel_system_roles.rhc` — An ansible role which connects RHEL systems to Red Hat. • `redhat.rhel_system_roles.timesync` — This role installs and configures an NTP and/or PTP implementation to operate • `redhat.rhel_system_roles.selinux` — Essentially provide mechanisms to manage local customizations: • `redhat.rhel_system_roles.firewall` — This role configures the firewall on machines that are using firewalld. • `redhat.rhel_system_roles.kernel_settings` — This role is used to modify kernel settings. For example, on Linux, settings • `redhat.rhel_system_roles.crypto_policies` — This Ansible role manages system-wide crypto policies. • `redhat.rhel_system_roles.logging` — Logging role is an abstract layer for provisioning and configuring the logging system. Currently, rsyslog is the only supported provider. • `ansible.posix.sysctl` — Manage sysctl entries
Notes	• Other roles in the collection cover `ad_integration`, `aide`, `auditd`, `bootloader`, `certificate`, `cockpit`, `fapolicyd`, `ha_cluster`, `journald`, `kdump`, `metrics`, `network`, `nbde_client`, `nbde_server`, `postfix`, `sudo`, `ssh`, `tlog` and `vpn`. Add them as the baseline grows. • See Manage users, sudo and SSH for access settings and Provision RHEL with Satellite for the build step that comes before this baseline. • Benchmark-driven hardening (CIS, DISA STIG) is a separate use case. Keep the SOE and the hardening profile as separate layers so each can change on its own schedule. • Use Event-Driven Ansible to re-run the baseline for a single host when a monitoring tool reports configuration drift.

Use Case: Manage users, sudo and SSH

Description	Make access to Linux hosts consistent and auditable. Identities come from a directory where possible; local accounts, SSH keys and sudo rules that remain are defined in Git and applied by Ansible, including the removal of access when people leave a team.
Collection	`redhat.rhel_system_roles` (Certified), `ansible.posix` (Certified), `redhat.rhel_idm` (Certified)
Key content	• `redhat.rhel_system_roles.sudo` — This role configures sudo. • `redhat.rhel_system_roles.ssh` — An Ansible role for managing ssh clients configuration. • `redhat.rhel_system_roles.ad_integration` — An ansible role which configures direct Active Directory integration. • `ansible.posix.authorized_key` — Adds or removes an SSH authorized key • `ansible.posix.acl` — Set and retrieve file ACL information.

Notes	• The `ssh` role configures the SSH client. Server-side `sshd` hardening is a separate task and belongs to the security hardening use cases. • The sudo role can replace the distribution `/etc/sudoers` and has an option to remove unlisted drop-in files. Test on one host first, and keep the break-glass account out of the change. • Store secrets such as the AD join password in an automation controller credential, not in variables. • Combine with Standardise RHEL configuration so access settings are part of the same baseline. • Windows equivalents are in Manage Active Directory.

Use Case: Remediate with Red Hat Insights

Description	Red Hat Insights finds configuration, vulnerability, patch and compliance issues on registered RHEL hosts and generates Ansible playbooks to fix them. This use case connects the hosts, builds inventories from Insights data and runs the fixes from Automation Platform with approval and reporting.
Collection	`redhat.insights` (Certified), `redhat.insights_eda` (Certified), `redhat.rhel_system_roles` (Certified), `redhat.satellite_operations` (Certified)
Key content	• `redhat.rhel_system_roles.rhc` — An ansible role which connects RHEL systems to Red Hat. • `redhat.insights.insights` — insights inventory source • `redhat.insights.compliance` — Install and configure Red Hat Insights Client • `redhat.insights_eda.insights` — Receive Red Hat Insights events. • `redhat.satellite_operations.cloud_connector` — Install and configure Red Hat Cloud Connector
Notes	• Use Patch RHEL at scale for planned, controlled patch cycles and this use case for targeted fixes of specific findings. • The `redhat.insights.insights_client` role is deprecated in favour of `redhat.rhel_system_roles.rhc`. • The `redhat.insights_eda` collection provides notification-driven events: advisor recommendations, new vulnerabilities and compliance below a threshold are examples in its documentation. • Insights remediation on a host needs the remediation packages installed; the `rhc` role installs them when remediation is enabled, which is the default, and the setting applies from RHEL 8.4.

Use Case: Convert CentOS and Oracle Linux to RHEL

Description	Move CentOS and Oracle Linux systems to supported RHEL without reinstalling them. The Convert2RHEL tool changes the operating system packages in place; Ansible runs a pre-conversion analysis on every host, fixes what blocks the conversion and then converts the hosts in controlled batches.
Collection	`infra.convert2rhel` (Validated), `redhat.satellite` (Certified), `infra.lvm_snapshots` (Validated)

Key content	• `infra.convert2rhel.analysis` — Tasks used for running convert2rhel analysis • `infra.convert2rhel.handle_c2r_report` — Tasks used for collecting and parsing convert2rhel analysis reports • `infra.convert2rhel.convert` — Tasks used to run a conversion through convert2rhel. • `redhat.satellite.convert2rhel` — Configure everything needed to register and convert CentOS clients to Red Hat Enterprise Linux • `infra.lvm_snapshots.snapshot_create` — Create a defined set of LVM snapshot volumes
Notes	• The collection `infra.convert2rhel` also ships ready-made playbooks `infra.convert2rhel.analysis` and `infra.convert2rhel.convert` that can be used as job template playbooks. • Plan a follow-up Upgrade RHEL in place with snapshot rollback if the converted release is not the target release. • Apply Patch RHEL at scale and Standardise RHEL configuration after conversion so the hosts join the managed estate. • Third-party agents and kernel modules from the old distribution often cause inhibitors. Identify them from the first analysis run.

Use Case: Upgrade RHEL in place with snapshot rollback

Description	Move RHEL hosts to the next major release without rebuilding them. Leapp analyses each host, Ansible fixes known blockers, takes an LVM snapshot, runs the upgrade and either removes the snapshot after sign-off or reverts to it if the application fails.
Collection	`infra.leapp` (Validated), `redhat.leapp` (Certified), `infra.lvm_snapshots` (Validated), `redhat.rhel_system_roles` (Certified)
Key content	• `infra.leapp.analysis` — Analysis phase of leapp • `infra.leapp.remediate` — Remedetation part of the leapp process • `infra.leapp.upgrade` — Upgrade phase of leapp • `infra.lvm_snapshots.snapshot_create` — Create a defined set of LVM snapshot volumes • `infra.lvm_snapshots.snapshot_revert` — Revert to snapshots previously created using the snapshot_create role • `infra.lvm_snapshots.snapshot_remove` — Remove snapshots previously created using the snapshot_create role • `infra.lvm_snapshots.bigboot` — Increase the size of the boot partition • `infra.lvm_snapshots.shrink_lv` — Decrease logical volume size along with the filesystem

Notes	• Build the flow as a workflow: analysis, remediation, approval, snapshot, upgrade, validation, then remove or revert as separate nodes with an approval before the destructive branch. • Do not migrate GRUB to GRUB 2 (`leapp_update_grub_to_grub_2`) in the same run as a snapshot. The snapshot role warns that it invalidates the boot backup and breaks a later revert. • `redhat.leapp` is the certified build of the same roles and is distributed through automation hub; `infra.leapp` is the validated-content build. `redhat.rhel_system_roles.snapshot` is a supported alternative for creating snapshots. • The `upgrade` role does not upgrade third-party products or non-RHEL repository content. Clustered storage, databases and agents may need separate automation. A rolling upgrade playbook for HA clusters, `infra.leapp.rolling_upgrade_ha_cluster`, is provided for HA clusters; it upgrades the Pacemaker CIB. • Related: Convert CentOS and Oracle Linux to RHEL and Patch RHEL at scale.

Product use cases

Grouped from the modules of each certified or validated collection. Key content lists the components each use case supports, with the modules that manage them (create, update, delete) and query them (_info).

POSIX and Linux systems `ansible.posix`

Certified Red Hat · version 2.2.2 · updated 2026-07-13 · 14 modules → 3 use cases

Use Case: Manage Linux system settings and security

Description	Manage SELinux state, booleans, sysctl entries, firewalld, file ACLs, mounts and SSH authorized keys.
Collection	`ansible.posix` (Certified, Red Hat)
Key content	**SELinux policy and state** `selinux` **SELinux booleans** `seboolean` **Kernel parameters** `sysctl` **firewalld** `firewalld`, `firewalld_info` **File ACLs** `acl` **Mount points** `mount` **SSH authorized keys** `authorized_key` **Scheduled commands** `at` **RHEL facts** `rhel_facts`

Use Case: Update files and image-based systems

Description	Copy files with rsync, apply patch files and manage packages and upgrades on rpm-ostree systems.
Collection	`ansible.posix` (Certified, Red Hat)

Key content	**rsync file synchronization** `synchronize` **Patch files** `patch` **rpm-ostree packages** `rhel_rpm_ostree` **rpm-ostree upgrades** `rpm_ostree_upgrade`

Use Case: Profile and format Ansible output

Description	Show task and role timing, cgroup performance, timers and JSON or debug-formatted output.
Collection	`ansible.posix` (Certified, Red Hat)
Key content	**Task timing** callback `profile_tasks` **Role timing** callback `profile_roles` **Play timer** callback `timer` **cgroup performance recap** callback `cgroup_perf_recap` **Debug output** callback `debug` **JSON output** callback `json` **JSONL output** callback `jsonl`

Red Hat convert2rhel `infra.convert2rhel`

Validated Red Hat · version 1.2.1 · updated 2024-07-03 · 0 modules → 1 use cases

Use Case: Analyse and convert systems to RHEL

Description	Run the convert2rhel pre-conversion analysis, collect and parse its reports and convert supported systems to Red Hat Enterprise Linux.
Collection	`infra.convert2rhel` (Validated, Red Hat)
Key content	**Analysis playbook** playbook `analysis` **Analysis** role `analysis` **Report handling** role `handle_c2r_report` **Conversion playbook** playbook `convert` **Conversion** role `convert` **Shared tasks** role `common`

RHEL in-place upgrade (Leapp) `infra.leapp`

Validated Red Hat · version 1.7.3 · updated 2026-05-05 · 2 modules → 2 use cases

Use Case: Analyze and remediate before the upgrade

Description	Run the pre-upgrade analysis and remediate the findings.
Collection	`infra.leapp` (Validated, Red Hat)

Key content	**Analysis phase** role `analysis` **Remediation phase** role `remediate` **Pre-upgrade analysis** playbook `analysis` **Remediation** playbook `remediate`

Use Case: Upgrade RHEL

Description	Run the upgrade phase, upgrade high availability clusters in a rolling fashion and support the process with logging, kernel module and RPM helpers.
Collection	`infra.leapp` (Validated, Red Hat)
Key content	**Upgrade phase** role `upgrade` **Logging** role `common` **RHEL upgrade** playbook `upgrade` **Rolling HA cluster upgrade** playbook `rolling_upgrade_ha_cluster` **Kernel modules** `modprobe` **Syslog messages** `sr_fingerprint` **RPM sorting** filter `rpm_sort`

Linux LVM `infra.lvm_snapshots`

Validated Red Hat · version 2.2.1 · updated 2026-07-21 · 0 modules → 1 use cases

Use Case: Snapshot, revert and resize LVM volumes

Description	Create, remove and revert LVM snapshots, shrink logical volumes and enlarge the boot partition.
Collection	`infra.lvm_snapshots` (Validated, Red Hat)
Key content	**Create snapshots** role `snapshot_create` **Remove snapshots** role `snapshot_remove` **Revert snapshots** role `snapshot_revert` **Shrink logical volumes** role `shrink_lv` **Grow boot partition** role `bigboot` **Temporary initramfs** role `initramfs`

RHEL image builder (osbuild) `infra.osbuild`

Validated Red Hat · version 2.3.0 · updated 2023-10-10 · 10 modules → 2 use cases

Use Case: Compose images from blueprints

Description	Create and push blueprints, add custom package repositories, start composes, wait for completion, export images and inject Kickstart files.
Collection	`infra.osbuild` (Validated, Red Hat)

Key content	**Blueprints** `create_blueprint` **Blueprint push** `push_blueprint` **Blueprint list** `list_blueprints` **Custom source repositories** `repository` **RHSM repositories** `rhsm_repo_info` **Composes** `start_compose` **Compose completion** `wait_compose` **Compose export** `export_compose` **Finished images** `get_all_finished_images` **Kickstart injection** `inject_ks` **Image builder** role `builder` **Run builder role** playbook `osbuild_builder`

Use Case: Set up build servers and update edge systems

Description	Set up the osbuild compose server, populate AAP with content, gather rpm-ostree system information and update systems.
Collection	`infra.osbuild` (Validated, Red Hat)
Key content	**Compose server** role `setup_server` **AAP content population** role `populate_aap` **rpm-ostree system information** role `system_info` **rpm-ostree updates** role `update_system` **Set up server** playbook `osbuild_setup_server` **Populate AAP** playbook `osbuild_populate_aap` **System information** playbook `osbuild_system_info` **Update systems** playbook `osbuild_update_system`

Red Hat Satellite `infra.satellite_configuration`

Validated Red Hat · version 1.4.0 · updated 2026-10-01 · 2 modules → 4 use cases

Use Case: Export Satellite configuration as code

Description	Export the live Satellite configuration to a file tree, scoped by organization and name filters.
Collection	`infra.satellite_configuration` (Validated, Red Hat)

Key content	**Configuration export** role `filetree_create` **Export Satellite configuration** playbook `run_filetree_create` **Export search query** filter `satellite_configuration_build_export_search_query` **Per-resource export search** filter `satellite_configuration_append_export_search_for_resource` **Export search on API path** filter `satellite_configuration_append_export_search_to_api_link` **Locked-object search on API path** filter `satellite_configuration_append_locked_export_search_to_api_link` **Export item filter** filter `satellite_configuration_filter_export_items` **Export organization filter** filter `satellite_configuration_filter_export_organizations` **Remaining export filters** filter `satellite_configuration_filters_for_post_api_export` **Locked object check** filter `satellite_configuration_export_item_is_locked` **Locked object names** filter `satellite_configuration_locked_item_names` **Locked object removal** filter `satellite_configuration_reject_locked_items` **Orphaned product cleanup** filter `satellite_configuration_reject_orphaned_katello_products` **Partition table sanitizing** filter `satellite_configuration_sanitize_partition_table_layout` **Raw marker removal** filter `satellite_configuration_unwrap_raw_markers` **Template marker detection** filter `satellite_configuration_contains_template_markers`

Use Case: Apply configuration as code to Satellite

Description	Read configuration files, resolve connection details and apply the definitions to Satellite.
Collection	`infra.satellite_configuration` (Validated, Red Hat)
Key content	**Read configuration files** role `filetree_read` **Apply configuration** role `dispatch` **Shared variables** role `global_vars` **Import into Satellite** playbook `run_filetree_read` **Override merging** filter `satellite_configuration_merge_by_key`

Use Case: Detect and reconcile configuration drift

Description	Compare live Satellite configuration with the code and write reconcile diff files.
Collection	`infra.satellite_configuration` (Validated, Red Hat)
Key content	**Configuration reconciliation** role `filetree_reconcile` **Reconcile Satellite** playbook `run_filetree_reconcile` **Reconcile diff files** `filetree_reconcile_diff`

Use Case: Format configuration files

Description	Format YAML files with PyYAML.
Collection	`infra.satellite_configuration` (Validated, Red Hat)
Key content	**YAML formatting** `format_yaml`

Windows Subsystem for Linux `microsoft.wsl`

Certified Red Hat · version 1.0.0 · updated 2026-07-29 · 8 modules → 1 use cases

Use Case: Manage WSL distributions

Description	Install, configure, import, export and run commands in WSL distributions, and report WSL status.
Collection	`microsoft.wsl` (Certified, Red Hat)
Key content	**Distributions** `distribution`, `distribution_info` **Distribution configuration** `distribution_config`, `distribution_config_info` **Import** `import` **Export** `export` **Commands** `command` **Status** `status_info`

OpenSVC `opensvc.cluster`

Certified OpenSVC · version 2.1.1 · updated 2026-10-12 · 0 modules → 2 use cases

Use Case: Build and remove clusters

Description	Build a cluster from a server inventory and remove clusters and their services.
Collection	`opensvc.cluster` (Certified, OpenSVC)
Key content	**Cluster provisioning** role `provision_cluster` **Cluster removal** role `unprovision_cluster` **Service removal** role `unprovision_services`

Use Case: Set up cluster networking, ingress and storage

Description	Install the backend network and CNI, name service, Envoy ingress gateway, DRBD prerequisites and storage pools.
Collection	`opensvc.cluster` (Certified, OpenSVC)

Key content	**Backend network** role `install_backend_network` **CNI plugins** role `install_cni` **Name service** role `install_dns` **Envoy ingress gateway** role `install_envoy` **DRBD prerequisites** role `install_drbd` **Storage pools** role `install_pool`

OpenSVC `opensvc.cluster_v3`

Certified OpenSVC · version 1.3.1 · updated 2026-10-12 · 0 modules → 2 use cases

Use Case: Build and remove clusters

Description	Build a cluster from a server inventory and remove clusters and their services.
Collection	`opensvc.cluster_v3` (Certified, OpenSVC)
Key content	**Cluster provisioning** role `provision_cluster` **Cluster removal** role `unprovision_cluster` **Service removal** role `unprovision_services`

Use Case: Set up cluster networking, storage and heartbeat

Description	Install the backend network and CNI, name service, DRBD prerequisites, storage pools and relay heartbeats.
Collection	`opensvc.cluster_v3` (Certified, OpenSVC)
Key content	**Backend network** role `install_backend_network` **CNI plugins** role `install_cni` **Name service** role `install_dns` **DRBD prerequisites** role `install_drbd` **Storage pools** role `install_pool` **Relay heartbeat** role `install_hb_relay`

Red Hat Insights `redhat.insights`

Certified Red Hat · version 1.3.1 · updated 2026-03-12 · 0 modules → 1 use cases

Use Case: Connect hosts to Red Hat Insights

Description	Install and configure the Insights client and compliance scanning, and use Insights as an inventory source.
Collection	`redhat.insights` (Certified, Red Hat)

Key content	**Insights client** role `insights_client` **Compliance** role `compliance` **Insights inventory** inventory `insights`

RHEL in-place upgrade (Leapp) `redhat.leapp`

Certified Red Hat · version 1.7.4 · updated 2026-07-20 · 2 modules → 2 use cases

Use Case: Analyze and remediate before the upgrade

Description	Run the pre-upgrade analysis and remediate the findings.
Collection	`redhat.leapp` (Certified, Red Hat)
Key content	**Analysis phase** role `analysis` **Remediation phase** role `remediate` **Pre-upgrade analysis** playbook `analysis` **Remediation** playbook `remediate`

Use Case: Upgrade RHEL

Description	Run the upgrade phase, upgrade high availability clusters in a rolling fashion and support the process with logging, kernel module and RPM helpers.
Collection	`redhat.leapp` (Certified, Red Hat)
Key content	**Upgrade phase** role `upgrade` **Logging** role `common` **RHEL upgrade** playbook `upgrade` **Rolling HA cluster upgrade** playbook `rolling_upgrade_ha_cluster` **Kernel modules** `modprobe` **Syslog messages** `sr_fingerprint` **RPM sorting** filter `rpm_sort`

Red Hat Enterprise Linux `redhat.rhel_system_roles`

Certified Red Hat · version 2.4.2 · updated 2026-10-06 · 0 modules → 6 use cases

Use Case: Harden system security

Description	Configure SELinux, crypto policies, auditing, file integrity monitoring, application allowlisting, sudo, SSH and remote attestation.
Collection	`redhat.rhel_system_roles` (Certified, Red Hat)

Key content	**SELinux** role `selinux` **Crypto policies** role `crypto_policies` **Audit rules** role `auditd` **AIDE intrusion detection** role `aide` **File access policy daemon** role `fapolicyd` **sudo** role `sudo` **SSH client** role `ssh` **Terminal session recording** role `tlog` **Keylime remote attestation** role `keylime_server` **Trustee server** role `trustee_server` **Trustee guest components** role `trustee_client`

Use Case: Manage networking and connectivity

Description	Configure network interfaces, firewalld, VPN tunnels, time synchronization and mail transfer.
Collection	`redhat.rhel_system_roles` (Certified, Red Hat)
Key content	**Network** role `network` **Firewall** role `firewall` **VPN tunnels** role `vpn` **Time synchronization** role `timesync` **Postfix mail** role `postfix`

Use Case: Manage storage and disk encryption

Description	Configure local storage, snapshots, GFS2 file systems and network-bound disk encryption.
Collection	`redhat.rhel_system_roles` (Certified, Red Hat)
Key content	**Local storage** role `storage` **Snapshots** role `snapshot` **GFS2 file systems** role `gfs2` **NBDE servers (Tang)** role `nbde_server` **NBDE clients (Clevis)** role `nbde_client`

Use Case: Tune and operate the base system

Description	Manage kernel settings, bootloader, kdump, systemd units, journald, logging, metrics and the Cockpit console.
Collection	`redhat.rhel_system_roles` (Certified, Red Hat)

Key content	**Kernel settings** role `kernel_settings` **Bootloader** role `bootloader` **kdump** role `kdump` **systemd units** role `systemd` **journald** role `journald` **Logging** role `logging` **Performance metrics** role `metrics` **Cockpit web console** role `cockpit`

Use Case: Integrate identity, certificates and Red Hat services

Description	Join Active Directory, issue and renew certificates, and connect systems to Red Hat.
Collection	`redhat.rhel_system_roles` (Certified, Red Hat)
Key content	**Active Directory integration** role `ad_integration` **Certificates** role `certificate` **Red Hat connector** role `rhc`

Use Case: Run applications, containers and clusters

Description	Manage Podman containers, PostgreSQL and high availability clusters.
Collection	`redhat.rhel_system_roles` (Certified, Red Hat)
Key content	**Podman** role `podman` **PostgreSQL** role `postgresql` **High availability clusters** role `ha_cluster`

Red Hat Satellite (Foreman and Katello) `redhat.satellite`

Certified Red Hat · version 5.11.0 · updated 2026-04-17 · 88 modules → 10 use cases

Use Case: Organise Satellite: organisations, locations and settings

Description	Create organisations and locations, tune Satellite settings and global parameters, and fetch server or task status.
Collection	`redhat.satellite` (Certified, Red Hat)

Key content	**Organizations** `organization`, `organization_info` **Locations** `location` **Settings** `setting`, `setting_info` **Global parameters** `global_parameter` **Bookmarks** `bookmark` **Server status** `status_info` **Task waits** `wait_for_task` **Resource information** `resource_info` **Organizations role** role `organizations` **Locations role** role `locations` **Settings role** role `settings`
Notes	• Modules are shared with upstream Foreman and Katello; roles configure Satellite end to end from variables.

Use Case: Manage users, roles and authentication

Description	Manage users, groups, roles, LDAP authentication sources, external user groups and webhooks.
Collection	`redhat.satellite` (Certified, Red Hat)
Key content	**Users** `user` **User groups** `usergroup` **Roles** `role` **External user groups** `external_usergroup` **LDAP authentication sources** `auth_source_ldap` **Webhooks** `webhook` **LDAP authentication sources role** role `auth_sources_ldap`
Notes	• Modules are shared with upstream Foreman and Katello; roles configure Satellite end to end from variables.

Use Case: Manage subscriptions and Red Hat content

Description	Upload subscription manifests, enable Red Hat repositories and define products, repositories and credentials, then sync them on a schedule.
Collection	`redhat.satellite` (Certified, Red Hat)

Key content	**Subscription manifests** `subscription_manifest` **Red Hat manifests** `redhat_manifest` **Subscriptions** `subscription_info` **Red Hat repository sets** `repository_set`, `repository_set_info` **Products** `product` **Repositories** `repository`, `repository_info` **Repository sync** `repository_sync` **Sync plans** `sync_plan` **Content credentials** `content_credential` **Content uploads** `content_upload` **Flatpak remotes** `flatpak_remote` **Flatpak remote scans** `flatpak_remote_scan` **Flatpak repository mirrors** `flatpak_remote_repository_mirror` **HTTP proxies** `http_proxy` **Candlepin labels** filter `cp_label` **Manifest role** role `manifest` **Repositories role** role `repositories` **Sync plans role** role `sync_plans` **Content credentials role** role `content_credentials`
Notes	• Modules are shared with upstream Foreman and Katello; roles configure Satellite end to end from variables.

Use Case: Promote content through lifecycle environments

Description	Build, filter, publish, version and clean up content views and promote them through lifecycle environments.
Collection	`redhat.satellite` (Certified, Red Hat)
Key content	**Lifecycle environments** `lifecycle_environment` **Content views** `content_view`, `content_view_info` **Content view filters** `content_view_filter`, `content_view_filter_info` **Content view filter rules** `content_view_filter_rule`, `content_view_filter_rule_info` **Content view versions** `content_view_version`, `content_view_version_info` **Content view history** `content_view_history_info` **Lifecycle environments role** role `lifecycle_environments` **Content views role** role `content_views` **Content view publishing role** role `content_view_publish` **Version cleanup role** role `content_view_version_cleanup`
Notes	• Modules are shared with upstream Foreman and Katello; roles configure Satellite end to end from variables.

Use Case: Export and import content between Satellites

Description	Export libraries, repositories and content view versions and import them on disconnected Satellites.
Collection	`redhat.satellite` (Certified, Red Hat)

Key content	**Content exports** `content_export_info` **Library exports** `content_export_library` **Repository exports** `content_export_repository` **Version exports** `content_export_version` **Content imports** `content_import_info` **Library imports** `content_import_library` **Repository imports** `content_import_repository` **Version imports** `content_import_version`
Notes	• Modules are shared with upstream Foreman and Katello; roles configure Satellite end to end from variables.

Use Case: Register and patch hosts

Description	Register hosts with activation keys, group them in collections and host groups, track errata, invoke remote execution jobs and convert CentOS to RHEL.
Collection	`redhat.satellite` (Certified, Red Hat)
Key content	**Activation keys** `activation_key` **Registration commands** `registration_command` **Hosts** `host`, `host_info` **Host collections** `host_collection` **Host groups** `hostgroup`, `hostgroup_info` **Host errata** `host_errata_info` **Host power state** `host_power` **Remote execution jobs** `job_invocation` **Job templates** `job_template` **Activation keys role** role `activation_keys` **Host groups role** role `hostgroups` **RHEL client registration and patching role** role `content_rhel` **CentOS to RHEL conversion role** role `convert2rhel`
Notes	• Modules are shared with upstream Foreman and Katello; roles configure Satellite end to end from variables.

Use Case: Provision hosts and manage infrastructure resources

Description	Define operating systems, installation media, partition and provisioning templates, images, compute resources and profiles, discovery rules and hardware models.
Collection	`redhat.satellite` (Certified, Red Hat)

Key content	**Operating systems** `operatingsystem` **Architectures** `architecture` **Installation media** `installation_medium` **Partition tables** `partition_table` **Provisioning templates** `provisioning_template` **Default template associations** `os_default_template` **Template imports** `templates_import` **Images** `image` **Compute resources** `compute_resource` **Compute profiles** `compute_profile` **Compute attributes** `compute_attribute` **Discovery rules** `discovery_rule` **Hardware models** `hardware_model` **Operating systems role** role `operatingsystems` **Provisioning templates role** role `provisioning_templates` **Compute resources role** role `compute_resources` **Compute profiles role** role `compute_profiles`
Notes	• Modules are shared with upstream Foreman and Katello; roles configure Satellite end to end from variables.

Use Case: Configure networking, Capsules and DNS

Description	Manage subnets, domains, realms and Capsules, and refresh Capsule features.
Collection	`redhat.satellite` (Certified, Red Hat)
Key content	**Subnets** `subnet`, `subnet_info` **Domains** `domain`, `domain_info` **Realms** `realm` **Capsules** `smart_proxy` **Capsule feature refresh** `smart_proxy_refresh` **Subnets role** role `subnets` **Domains role** role `domains`
Notes	• Modules are shared with upstream Foreman and Katello; roles configure Satellite end to end from variables.

Use Case: Manage Puppet and compliance content

Description	Manage Puppet environments, classes, config groups and smart class parameters, and SCAP content and tailoring files.
Collection	`redhat.satellite` (Certified, Red Hat)
Key content	**Puppet environments** `puppet_environment` **Puppet class imports** `puppetclasses_import` **Config groups** `config_group` **Smart class parameters** `smart_class_parameter` **Parameter override values** `smart_class_parameter_override_value` **SCAP content** `scap_content` **SCAP tailoring files** `scap_tailoring_file`
Notes	• Modules are shared with upstream Foreman and Katello; roles configure Satellite end to end from variables.

Use Case: Integrate with Foreman inventory and events

Description	Use Foreman as a dynamic inventory source and send Ansible events to Foreman.
Collection	`redhat.satellite` (Certified, Red Hat)
Key content	**Foreman inventory** inventory `foreman` **Foreman callback** callback `foreman`
Notes	• Modules are shared with upstream Foreman and Katello; roles configure Satellite end to end from variables.

Red Hat Satellite `redhat.satellite_operations`

Certified Red Hat · version 4.0.0 · updated 2026-03-24 · 0 modules → 1 use cases

Use Case: Install, back up and monitor Red Hat Satellite

Description	Run the Satellite installer, generate Capsule certificate bundles, schedule backups, connect to Red Hat Cloud and gather performance metrics with Performance Co-Pilot and Grafana.
Collection	`redhat.satellite_operations` (Certified, Red Hat)
Key content	**Satellite installer** role `installer` **Installer RPM placeholder** role `fake_installer_rpm` **Capsule certificate bundles** role `capsule_certs_generate` **Backup cron job** role `backup` **Red Hat Cloud Connector** role `cloud_connector` **Performance metrics (PCP and Grafana)** role `metrics`